ABRA — Project Summary

Version 2.4.0 · 2026-07-23 · Will Hooper

A one-page map of the whole project and every component. For depth: the white paper (math + sources), the deck (plain-English), the technical docs (how to run it), and the living model ledger.

What ABRA is

ABRA is a decision-support model family for **Pokémon Champions VGC, Reg M-B, best-of-one closed-sheet ladder**. It stores every public ladder replay and builds small, CPU-trainable models on that growing store. It runs in a browser with no build step.

The finding that shapes everything

You cannot reliably predict who wins from the two team sheets — even a player-rating model ties a coin. So ABRA does not sell outcome prediction. It supports *decisions* and grades every model with a proper score, a confidence interval, and an honest baseline. Wins are reported as wins; two honest negatives are reported as negatives.

The components at a glance

Model	What it is	Status	Headline result
MEDICHAM	Exact Gen-9 doubles damage engine	☐Validated	Within 5% of `@smogon/calc` on 100% of 31 scenarios (median 0%)
GURU	Meta matchup matrix from real outcomes	☐Built	13 archetypes over 5,199 games, Wilson CIs (descriptive; winner-prediction ties a coin, as expected)
XATU	Opponent set + next-move belief	☐Built	Top-1 36% / top-3 72% on held-out human moves (beats its baselines)
PORY	Mid-game win-probability value net	☐**The win**	Log-loss 0.567 vs coin 0.693, calibrated (ECE 1.6%); live in KADABRA
CHOMP	Bring-4 / lead-2 team-preview engine	☐Ships (standalone)	Exact-damage picker; **CHOMP-EV proof: brings tie a coin (honest null)**
SLOWKING	Team-preview Nash (mixed strategy)	☐Built	Equilibrium ≪ exploitable than uniform; playstyle cycle is **suggestive on small samples**
KADABRA	Replay coach	☐Works offline	Per-turn "you're at X%" from PORY
DITTO	Team optimiser	⚠ Pivoting	Objective de-biased to validated damage (was optimising a backwards signal)
ALAKAZAM	In-battle decision engine (capstone)	☐In development	Belief + search + learned value; built last on the inputs above

Model	What it is	Status	Headline result
MEW	Self-play data engine	□Roadmap	The path to the millions of games ALAKAZAM's strongest version needs

How it fits together

The **store** (every real game) feeds **GURU** (meta), **XATU** (belief), **PORY** (value), and **MEDICHAM** (damage). **SLOWKING** solves the preview; **CHOMP** picks the bring; **KADABRA** coaches a replay with PORY. **ALAKAZAM** is the capstone that assembles belief + search + value into the win-%-optimal move, built last. Every change updates the code, this summary, the white paper, the deck, the technical docs, and the CHANGELOG in the same pass.

Repositories and site

Piece	Repo	Live
ABRA (models + site)	`github.com/willhoop/abra`	`willhoop.github.io/abra/app/`
CHOMP (bring engine)	`github.com/willhoop/chomp`	Showdown userscript
Portfolio	`github.com/willhoop/willhoop.github.io`	`willhoop.github.io`

Honest ceilings

Predicting the match winner from sheets is a permanent coin flip in this format. The meta-structure models (playstyle, cores) are small-sample and stay suggestive until the store grows (~18k games/week). The load-bearing wins are the validated damage and the mid-game value net.

ABRA — the plain-English deck

Version 2.4.0 · 2026-07-23 · Will Hooper

A slide-by-slide, jargon-light tour. The white paper (linked on the last slide) has the math and sources.

Slide 1 — ABRA

A family of small AI models for competitive Pokémon (Champions VGC). Built from thousands of real ladder games. Honest about what it can and can't know.

Slide 2 — The big idea

You **can't reliably predict who wins** a match from the two team sheets — in this format it's close to a coin flip, and even a strong player-rating model barely beats a coin. So ABRA stops trying to call the winner and instead **helps you make better decisions**: what to bring, and what to do turn by turn.

This is the same move sports analytics made with "expected goals" — you can't predict the final score, but you can measure the value of each shot. ABRA is that idea, pointed at Pokémon.

Slide 3 — How it's built

One rule: **keep every real game, and analyse on top of it.** ABRA saves every public ladder replay into a growing library and builds its models on that library — so it gets smarter as more games come in, and never has to re-download anything. It runs on a normal laptop; no special hardware.

Slide 4 — The town of models

Each model is a "house" you can visit on the site:

- **MEDICHAM** — the damage engine. Knows exactly how hard every move hits. (Checked against the community standard and matches it.)
- **GURU** — reads the metagame: which team styles beat which, from real results, with error bars.
- **XATU** — reads the opponent: the likely item, ability, and moves behind each Pokémon.
- **PORY** — the win meter: mid-battle, tells you your real chance to win.
- **SLOWKING** — the strategist: there's no single best team, so it plays a smart mix.
- **CHOMP** — the team picker: which four to bring, which two to lead.
- **ALAKAZAM** — the coach (in progress): the best move to make, right now.

Slide 5 — The win

PORY beats a coin flip. Mid-game, using how many Pokémon each side has left and their health, it calls your win chance far better than chance — and it's *calibrated*, meaning when it says 70% it's really about 70%. It's live on the site as a per-turn "you're at X%".

Slide 6 — The honest parts (this is a feature)

Two things we tested and reported straight, even though they're negatives:

- **Picking the team doesn't beat a coin.** We tested whether the team-picker's choices track who wins — they don't, more than chance. So we don't oversell it. The damage math is still exact and useful.
- **The "rock-paper-scissors" metagame is a hint, not a fact.** The data suggests Trick Room beats Hyper Offense beats Sand beats Trick Room — but each of those rests on only ~13–18 games, so the error bars are wide. We label it suggestive, and it'll firm up as more games arrive.

Saying what we *can't* prove, as plainly as what we can, is the whole point.

Slide 7 — What's next

- **ALAKAZAM** — the in-battle coach. A light version runs in your browser; a version that could beat top humans needs a rented cloud computer and months of the AI playing itself to learn.
- **A million games.** Real games trickle in ~18k/week; the AI playing itself can generate millions fast, which is what the coach needs to get truly strong.

Slide 8 — Read more

Full technical detail, math, and sources: **ABRA white paper**. Also: **project summary** · **technical docs** · **model ledger**.

Supporting Decisions in a Near-Unpredictable Game

A technical description of ABRA, a decision-support model family for competitive Pokémon

Version 2.4.0 · Last updated 2026-07-23 Will Hooper · ABRA

> This is a living document, updated in the same pass as any change to the code, together with the deck and the technical documentation. A prior conclusion is never silently rewritten; new information is added and what changed is stated. See `CHANGELOG.md`.

Abstract

ABRA is a decision-support model family for **Pokémon Champions VGC, Regulation M-B, best-of-one closed-sheet ladder**. It continuously ingests public battle replays from Pokémon Showdown and stores the durable facts of every game, then builds small, CPU-trainable models on that store. Its central empirical finding governs its design: **predicting the winner of a game from the two team sheets is near-impossible in this format — even a player-Elo model ties a coin.** ABRA therefore does not sell outcome prediction. It follows the recipe that worked in poker, Diplomacy, and sports analytics: *support decisions, don't predict outcomes*, and judge every model by a proper score against an honest baseline with a confidence interval. This paper states the empirical ceiling, the data model, each model with its validated result (including two honest negatives), the mathematics, the limits, and the road to the in-battle engine (ALAKAZAM).

1. The empirical ceiling (why the design is what it is)

On 600+ held-out real Champions games, a Bradley-Terry player-Elo model reaches a held-out log-loss of **0.687 against a coin's 0.693** — a real but negligible edge. A cloned-policy rollout engine (MEDICHAM) does *worse* than a coin as a raw win-predictor (log-loss ≈ 1.2; it picks the actual winner on ~44% of decisive calls, i.e. it is systematically *inverted*, over-backing fast offensive teams that lose more). After held-out Platt recalibration it only edges the coin (0.6897 vs 0.6931).

The conclusion is not "our models are weak." It is a property of the game: a two-player, zero-sum, **imperfect-information, simultaneous-move** game with a non-transitive metagame has an irreducible outcome-prediction ceiling from team sheets alone. This is the same reason expected-goals (xG) models in football predict *shot quality* rather than final scores. **Design consequence: stop predicting outcomes; support decisions.** Everything below serves that.

2. Data: store raw, analyse on top

ABRA reads Showdown's public replay API (`search.json?format=` , `search.json?user=` , `<id>.log`); it reads nothing private and creates no accounts (`SECURITY.md`). The extractor (`engine/durable-ingest.js` , `extract()`) turns one battle log into one durable record:

Field	Meaning
`id` , `date`	replay id and upload time
`p1` , `p2`	`{name, rating, bot}` per player
`six.p1/p2`	the revealed team of six
`brought.p1/p2`	the four actually brought
`lead.p1/p2`	the two led
`sets`	per species, the moves / item / ability the replay *revealed*
`turns`	per-turn events (moves, damage, faints, status, field)
`winner`	the winning name

The store is append-only JSON Lines keyed by replay id: idempotent, deduplicated at read time, and grown hourly by a GitHub Action. The **governing rule** is *store raw, analyse on top*: every filter (rating tier, humans-only, archetype, playstyle) is a re-computation over the store, never a re-pull. Changing how we segment games is free; the fetch is a one-time cost. About 2,600 public games/day are available, and the store grows ~18k/week, so every model below sharpens on its own over time.

3. The validated foundation — exact damage (MEDICHAM)

The one component that is *not* a coin flip is the damage engine. MEDICHAM's Gen-9 doubles damage pipeline (`engine/medicham2-browser.js`) is validated against `@smogon/calc` (the community ground-truth) on 31 meta scenarios: **within 5% on 100% of scenarios, median error 0%, worst 3%** (16-roll rounding). This is gated in CI (`engine/validate_damage.js` → `data/damage-validation.json`). Every model that reasons about damage builds on this, and "will this move KO?" is a *winnable* prediction, unlike "who wins the game."

4. The models and their validated results

Every probability ships a **proper score** (log-loss and/or Brier), a **confidence interval** (clustered by game where states within a game are correlated), and an **honest baseline**, persisted to JSON and gated in CI.

4.1 GURU — meta / matchup matrix (descriptive)

From REAL outcomes, `engine/guru.py` builds a 13-archetype × 13-archetype matchup matrix over 5,199 games, each cell a win-rate with a **Wilson score interval**. GURU is *descriptive*: its own predictive test shows per-game winner prediction from the matrix ties a coin (log-loss 0.7122 vs

0.6931), exactly as §1 predicts. Its value is honest matchup structure with error bars, and it is the real (not simulated) payoff matrix that SLOWKING solves. Output: `data/guru-matchups.json`, `data/guru.js`.

4.2 XATU — opponent belief (modest, useful)

`engine/xatu.py` learns, per species, the set (item/ability/moves) usually run, and predicts the opponent's next move from state. On held-out human moves the behaviour-clone reaches **top-1 35.9% (CI 35.2–36.5), top-3 71.6%**, cross-entropy 2.27 nats — beating a species-agnostic baseline (4.54) and uniform-over-moveset (2.91). A modest but real signal; human move choice has genuine entropy. Output: `data/xatu.json`, `data/xatu.js`; harness `engine/eval_policy.py` → `data/policy-eval.json`.

4.3 PORY — mid-game win-probability value net (the win)

The pivot's proof. `engine/pory.py` reconstructs per-turn board state (mons alive out of four, mean active HP, turn) and fits a logistic value net. Held-out, clustered by game: **log-loss 0.567 vs coin 0.693**, beating a material-sign heuristic, **calibrated to ECE 1.6%**, CI **[0.548, 0.583]**. The *live board is predictable even though the pre-game sheets are not* — the thesis, demonstrated. PORY is wired into KADABRA as a per-turn "you're at X%". Output: `data/pory.js`; report `data/pory-eval.json`.

4.4 CHOMP-EV — do CHOMP's brings beat humans'? (honest NULL)

The winnable team-preview test. For each held-out game (both full sixes, both actual brings, the winner), `engine/chomp_ev.js` ranks each side's *actual* bring among all 15 candidate brings by CHOMP's exact-damage coverage, and asks whether that quality signal tracks who won. On **1,205 games**: CHOMP's bring ranking **does not beat a coin** (held-out log-loss 0.6918 vs 0.6931, CIs overlap), ties an Elo and a usage-prior baseline, and winners are only marginally more CHOMP-aligned than losers (sign test 0.512, CI [0.493, 0.535]). It is **robust to forfeits** (0.505; a forfeit is usually a concession from a losing position, and dropping all forfeits does not change the result), and a measured **selection audit** shows the required "all four revealed" filter is a mild bias (eval 6.5 turns / 1280 rating vs 6.08 / 1267 excluded) that, if anything, *favours* CHOMP — making the null conservative. A **belief-weighted** variant (coverage vs the opponent's likely-4) also ties the coin (0.6924). Interpretation: the bring decision sits at the same near-coin ceiling as pre-game prediction; CHOMP's damage math stays validated and useful as a calculator, but "CHOMP builds better brings" is not yet empirically supported. This negative is a guardrail: it stops optimising a bring metric that carries no held-out winning signal (a Goodhart trap). Report `data/chomp-ev.json`; test `tests/test-chomp-ev.js`.

4.5 SLOWKING — team-preview Nash and the playstyle cycle (suggestive)

`engine/slowking_preview.py` solves a matchup matrix to a mixed-strategy equilibrium and grades it by **exploitability** (the worst-case win-edge a best pure counter extracts; lower is better; Nash ≈ 0), against greedy "single best deck" and uniform baselines, with a bootstrap CI that propagates matchup-count uncertainty (Beta resampling). Over GURU's 13 species-archetypes the equilibrium is far less exploitable than uniform (Nash ≈ 0 vs 0.109), but greedy ≈ Nash because this meta is currently near-transitive (a dominant deck). A **playstyle** re-analysis (`engine/`

`playstyle.js` classifies each team as TrickRoom / Rain / Sun / Sand / Snow / Setup / PerishTrap / TailwindOffense / FakeOutBalance / Stall / HyperOffense) surfaces a non-transitive cycle — **TrickRoom → HyperOffense → Sand → TrickRoom** — with a point exploitability gap of ~0.073 for greedy; the equilibrium now correctly leads with Sun (~31%), since Reg M-B Charizard is Mega-Y (Drought) and is classified as a Sun setter. **Honest caveat:** each cycle leg rests on only 13–18 games (win rates 73% / 71% / 67%) with 95% CIs that cross 50%, so the cycle is a **suggestive pattern, not a settled fact**; it will sharpen as the store grows. Where matchups *are* well-sampled they tend to run flat against intuition — **Rain vs Sun is 51% (n=236)** and **Tailwind vs no-Tailwind is 47% (n=756)**, both statistical coin-flips. Reports `data/slowking-eval.json`, `data/slowking-playstyle-eval.json`; test `tests/test-slowking.py`.

5. Mathematics

Wilson score interval (used for every matchup rate) for `w` wins in `n` games, `z = 1.96`: `(p̂ + z²/2n ± z·√(p̂(1−p̂)/n + z²/4n²)) / (1 + z²/n)`, with `p̂ = w/n`. It is well-behaved at small `n` and near 0/1, unlike the normal approximation.

Value net. Features `x = [alive_diff, hp_diff, my_alive, foe_alive, turn/10]` are standardised by train mean/std; `P(win) = σ(w·z + b)`. Graded by held-out **log-loss** `−(y·ln p + (1−y)·ln(1−p))` and **Brier** `(p−y)²`; the coin scores `ln 2 = 0.6931` and `0.25` respectively.

Equilibrium and exploitability. Each preview is a two-player zero-sum matrix game on an antisymmetric edge matrix `M[i,j] = (p(i>j) − p(j>i))/2`. Regret matching (Hart & Mas-Colell) converges to an ε-Nash. For a strategy `x`, **exploitability** `= −minⱼ (x·M[:,j])` — the worst-case loss to a best response; the Nash value is 0, so a Nash strategy scores ≈ 0 and a predictable single-deck strategy is punished.

Confidence intervals. Because per-turn states within a game are correlated, CIs are **bootstrapped by resampling games** (clustered), not states. Matchup-matrix uncertainty is propagated by **Beta(n·p+1, n·(1−p)+1) resampling** of each cell before re-solving.

Future rating math. For any descriptive meta-rating we will use an **intransitivity-capable** class (blade-chest / low-rank bilinear, Chen & Joachims 2016; or Nash-averaging, Balduzzi et al. 2018) and a **Helmholtz–Hodge / HodgeRank** decomposition (Jiang, Lim, Yao & Ye 2011) to split the matchup flow into a transitive ranking plus a cyclic (rock-paper-scissors) component — the correct tool for "which cores beat which" and for quantifying how cyclic the meta really is.

6. Limitations and honest ceilings

1. **The game-winner ceiling is permanent** (Elo ≈ coin). SLOWKING/ALAKAZAM are judged on decision quality and self-play/ladder win-rate, never on match-outcome prediction.
2. **Revealed sets are partial** (a mon that never attacked reveals no moves); belief is a lower bound.
3. **Small samples in the meta layer.** Playstyle and core matchups are thin; those results are suggestive until the store grows.

4. **Policy is the residual GIGO.** The damage is validated; the rollout *policy* is behaviour-cloned and over-credits speed control. The learning path (PORY, ALAKAZAM) partly sidesteps this by learning from real outcomes.
5. **Champions rule specifics** (sleep/paralysis edge cases) are flagged, not yet fully modelled.

7. The road to ALAKAZAM

ALAKAZAM is the in-battle capstone, built last on the inputs above. Given a live position it will output the win-%-optimal move (a mixed strategy) and its value by: (1) a **belief** over the opponent's hidden sets (XATU), updated by a Bayesian filter; (2) **depth-limited search** over the validated damage engine, solving each simultaneous turn as a **matrix game** (regret matching — this removes the speed bias that inverted the greedy engine); (3) a **learned value** at the leaves (PORY, grown to an NNUE-style net); (4) **human-anchoring** (KL-regularised to the behaviour-clone) so it stays strong and unexploitable. Inference is light (CPU / Web Worker / WASM); the strongest version needs offline RL on millions of human + self-play games and a rented cloud GPU. It is judged on decision quality and self-play/ladder win-rate with CIs — never on predicting the winner. A self-play data engine (MEW) is the pacing item toward the millions of games that path needs.

8. References

1. Zinkevich et al., *Regret Minimization in Games with Incomplete Information* (CFR), 2007.
2. Lanctot et al., *Monte Carlo Sampling for Regret Minimization* (MCCFR), 2009.
3. Moravčík et al., *DeepStack*, Science 2017. · Brown & Sandholm, *Libratus*, Science 2018.
4. Brown et al., *Combining Deep RL and Search* (ReBeL), NeurIPS 2020. · Schmid et al., *Player of Games*, 2021.
5. Angliss et al., *VGC-Bench*, arXiv 2506.10326, 2025. · UT-Austin-RPL, *Metamon* (offline RL + transformers), arXiv 2504.04395, 2025.
6. Perolat et al., *DeepNash / R-NaD* (Stratego), Science 2022. · Vinyals et al., *AlphaStar*, 2019.
7. Meta FAIR, *CICERO / piKL* (human-regularised RL, Diplomacy), Science 2022.
8. Chen & Joachims, *Modeling Intransitivity in Matchup Data* (blade-chest), WSDM 2016. · Balduzzi et al., *Re-evaluating Evaluation* (Nash-averaging), NeurIPS 2018.
9. Jiang, Lim, Yao & Ye, *Statistical Ranking and Combinatorial Hodge Theory* (HodgeRank), 2011.
10. Wilson, *Probable Inference, the Law of Succession, and Statistical Inference*, JASA 1927.
11. `@smogon/calc` — community damage ground-truth. · Pokémon Showdown replay API.

Companion documents. Slide deck · Technical documentation · Model ledger · Changelog

ABRA — Technical Documentation

Version 2.4.0 · Last updated 2026-07-23

Written in ASD-STE100 Simplified Technical English. Sentences are short. The voice is active. One word has one meaning. The document follows the Diátaxis structure: Tutorial, How-to, Reference, Explanation.

---

1. Tutorial — run ABRA for the first time

Do these steps in order.

1. Get the code. Clone the repository `github.com/willhoop/abra`.
2. Get the sibling engine. Clone `github.com/willhoop/chomp` next to it. ABRA reads it at `../CHOMP`.
3. Open the site. Open `web/index.html` in a web browser. The site needs no build step and no server.
4. Visit a model. Click a house in the town. Open **PORY** and move the sliders. The win % updates.
5. Run one check. In a terminal, run `node engine/validate_damage.js`. It confirms the damage engine.

You now have the site and one validated model.

2. How-to — common tasks

Pull new replays. `PAGES=6 CONC=20 node engine/durable-ingest.js data/games.ladder.jsonl` The command adds only new games. It never duplicates a game and never re-fetches a stored game.

Rebuild the meta model. `node engine/analyze.js data/games.ladder.jsonl` writes `data/meta-usage.json`.

Refresh the site data. `python3 engine/refresh-site-data.py` writes `data/live.js`, `data/archetypes.json`, and `data/kad-replays.js`.

Run a model or an evaluation.

- GURU matchup matrix: `python3 engine/guru.py`
- XATU belief / policy eval: `python3 engine/eval_policy.py`
- PORY value net: `python3 engine/pory.py`
- CHOMP-EV proof: `node engine/chomp_ev.js`
- SLOWKING preview-Nash (species): `python3 engine/slowking_preview.py`
- SLOWKING preview-Nash (playstyle): first `node engine/playstyle.js`, then `MATRIX_FILE=data/playstyle-matchups.json TAG=playstyle python3 engine/slowking_preview.py`

Validate the damage engine. `node engine/validate_damage.js`. It fails if any scenario is more than 5% from `@smogon/calc`.

Run the tests. `node tests/test-parse.js`, `node tests/test-dynamics.js`, `node tests/test-medicham.js`, `node tests/test-chomp-ev.js`, `python3 tests/test-jolteon.py`, `python3 tests/test-slowking.py`.

Edit the site, then mirror it. After you change `web/index.html`, copy it: `cp web/index.html app/index.html`.

3. Reference

3.1 Stored game record (`data/games.ladder.jsonl`, one JSON object per line)

Field	Meaning
`id`, `date`	replay id and upload time
`p1`, `p2`	`{name, rating, bot}` per player
`six.p1/p2`	the six revealed at preview
`brought.p1/p2`	the four actually brought
`lead.p1/p2`	the two led
`sets`	per species, the revealed moves / item / ability
`turns`	per-turn events (move, damage, faint, status, field)
`winner`	the winning name

3.2 Model outputs

File	Written by	Contents
`data/damage-validation.json`	`validate_damage.js`	damage error vs `@smogon/calc`
`data/guru-matchups.json`, `guru.js`	`guru.py`	archetype matchup matrix, Wilson CIs
`data/xatu.json`, `xatu.js`	`xatu.py`	opponent set/move belief
`data/pory.js`, `pory-eval.json`	`pory.py`	mid-game value net + its score
`data/chomp-ev.json`	`chomp_ev.js`	the bring proof (null result)
`data/playstyle-matchups.json`	`playstyle.js`	playstyle matchup matrix
`data/slowking-eval.json`, `slowking-playstyle-eval.json`, `slowking*.js`	`slowking_preview.py`	preview equilibrium + exploitability

File	Written by	Contents
`data/meta-usage.json`, `live.js`	`analyze.js`, `refresh-site-data.py`	usage model + live site counts

3.3 Continuous collection

A GitHub Action (`.github/workflows/ingest.yml`) runs the pull hourly and commits the store. A separate tests workflow runs the test suite and the damage validation on every push and pull request.

4. Explanation

Store raw, analyse on top. ABRA saves every game with every fact it may ever need. Every filter — rating tier, humans-only, archetype, playstyle — runs over the store at read time. A change to how the games are segmented is a re-computation, not a re-download. This makes the fetch a one-time cost and keeps the analysis free to change.

Support decisions, do not predict outcomes. In this format, the winner of a game is near-impossible to predict from the two team sheets; even a player-rating model ties a coin. ABRA therefore judges each model on a decision, not on the match result. Each probability ships a proper score (log-loss or Brier), a confidence interval, and an honest baseline.

Why the confidence interval is clustered. States within one game are correlated. A confidence interval that resamples states would be too narrow. ABRA resamples whole games instead. For a matchup matrix, it also resamples each cell from its Beta distribution before it solves, so the interval carries the small-sample uncertainty.

Honest negatives are kept. Two results are negative and are reported plainly: the team-picker's brings do not beat a coin (CHOMP-EV), and the playstyle rock-paper-scissors cycle rests on small samples and is suggestive, not settled. A negative that is measured is more useful than a positive that is asserted.

Companion documents. White paper · Deck · Project summary · Model ledger · Changelog

ALAKAZAM

The in-battle coach — a plain brief

The final, hardest piece of ABRA, a system that learns competitive Pokémon from thousands of real games.

The problem

A competitive Pokémon battle has two hard properties at once. First, **hidden information**: you can't see the opponent's full setup — like a hidden hand in poker. Second, **same-time choices**: both players pick their move for the turn at the same moment and reveal together — like rock-paper-scissors, not like poker's take-turns betting. Wrap that around a board that changes like chess, and the winning play often isn't one fixed move; it's a smart, slightly unpredictable mix.

What ALAKAZAM will be

A coach that, at any point in a battle, tells you the best move to make and how likely you are to win — the equivalent of a chess engine's "best move + evaluation," but for this hidden, same-time game.

How it works

- **A read on what's hidden** — a running best-guess of the opponent's setup, sharpened as the game reveals clues.
- **Looking ahead** — it imagines the next few turns (not the whole game, which is far too big) and scores where each line leads.
- **A fast judge of positions** — a small learned model that estimates "how likely am I to win from here," so it need not play every line to the end.
- **Staying human-like** — kept close to how real players play, so it's strong without drifting into weird, easily-punished habits.

Compute needs

Using it is light — it runs in a web browser on a normal laptop, no special hardware. Building the strongest version is the heavy part: it learns from a huge pile of past matches and from playing millions of games against itself, which needs a powerful rented cloud computer (a GPU) for a while.

How long

A basic but genuinely useful version: about **one to two weeks**. A version that could beat top human players: **a few months** — and most of that time is spent letting it play itself to learn, not the training run itself.

The honest limit

Some things simply can't be predicted — like who wins from the starting line-ups alone, which is close to a coin flip. So ALAKAZAM is judged on making **good decisions in the moment**, not on calling the winner in advance.